Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5
По дисциплине: «интеллектуальный анализ данных»
Тема: «Деревья решений»

Выполнил:
Студент 4 курса
Группы ИИ-25
Пальчастый Я.М
Проверил:
Андренко К.В.

Брест 2026

Цель работы: На практике сравнить работу нескольких алгоритмов одиночного дерева решений, случайного леса и бустинга для деревьев решений.

Вариант 8

- Seeds
- Классифицировать семена на три сорта пшеницы (Kama, Rosa, Canadian) на основе их геометрических параметров
- **Задания:**
  1. Загрузите и стандартизируйте данные;
  2. Разделите выборку на обучающую и тестовую;
  3. Обучить на обучающей выборке одиночное дерево, случайный лес и реализовать бустинг для решающих деревьев (AdaBoost, CatBoost, XGBoost);
  4. Сравните общую точность (accuracy) всех трех моделей;
  5. Визуализируйте данные в 2D (например, с помощью PCA), раскрасив точки в соответствии с предсказаниями лучшей модели.

Код программы:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from catboost import CatBoostClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt


data = pd.read_csv('seeds_dataset.txt', sep=r'\s+', header=None)
data.columns = ['area', 'perimeter', 'compactness', 'length',
                'width', 'asymmetry', 'groove', 'class']
data['class'] = data['class'] - 1
imputer = SimpleImputer(strategy='mean')
X = data.drop('class', axis=1)
y = data['class']
X_imputed = imputer.fit_transform(X)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_imputed)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42, stratify=y
)


models = {
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'AdaBoost':      AdaBoostClassifier(n_estimators=50, random_state=42),
    'CatBoost':      CatBoostClassifier(iterations=500, verbose=0, random_state=42),
    'XGBoost':       XGBClassifier(n_estimators=200, learning_rate=0.1,
                                   max_depth=3, eval_metric='mlogloss',
                                   random_state=42),
}
```

```
results = {}
preds_all = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred_test = model.predict(X_test)
    results[name] = accuracy_score(y_test, y_pred_test)
    preds_all[name] = model.predict(X_scaled)
    print(f'{name:15s}: accuracy = {results[name]:.4f}')


pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
ev = pca.explained_variance_ratio_
print(f'\nОбъяснённая дисперсия: PC1 = {ev[0]:.3f}, PC2 = {ev[1]:.3f}, '
      f'сумма = {ev.sum():.3f}')

fig, axes = plt.subplots(2, 3, figsize=(16, 10))
axes = axes.flatten()

for i, (name, y_pred) in enumerate(preds_all.items()):
    ax = axes[i]
    sc = ax.scatter(X_pca[:, 0], X_pca[:, 1], c=y_pred,
                    cmap='viridis', edgecolor='k', s=45, alpha=0.85)
    ax.set_title(f'{name}\naccuracy = {results[name]:.4f}', fontsize=11)
    ax.set_xlabel(f'PC1 ({ev[0]*100:.1f}%)')
    ax.set_ylabel(f'PC2 ({ev[1]*100:.1f}%)')
    ax.grid(True, linestyle='--', alpha=0.5)

# шестая ячейка — истинные метки для сравнения
ax = axes[5]
sc = ax.scatter(X_pca[:, 0], X_pca[:, 1], c=y,
                cmap='viridis', edgecolor='k', s=45, alpha=0.85)
ax.set_title('Истинные классы', fontsize=11)
ax.set_xlabel(f'PC1 ({ev[0]*100:.1f}%)')
ax.set_ylabel(f'PC2 ({ev[1]*100:.1f}%)')
ax.grid(True, linestyle='--', alpha=0.5)


cbar = fig.colorbar(sc, ax=axes, fraction=0.02, pad=0.02, ticks=[0, 1, 2])
cbar.ax.set_yticklabels(['0 (Kama)', '1 (Rosa)', '2 (Canadian)'])

plt.suptitle('PCA-проекция: предсказания каждой модели vs истинные классы',
             fontsize=14, y=0.98)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig('pca_all_models.png', dpi=150, bbox_inches='tight')
plt.show()


plt.figure(figsize=(9, 5))
names = list(results.keys())
accs  = list(results.values())
colors = ['#4C72B0', '#DD8452', '#55A868', '#C44E52', '#8172B2']

bars = plt.bar(names, accs, color=colors, edgecolor='black')
for bar, acc in zip(bars, accs):
    plt.text(bar.get_x() + bar.get_width() / 2, acc + 0.005,
             f'{acc:.4f}', ha='center', va='bottom', fontweight='bold')

best = max(results, key=results.get)
bars[names.index(best)].set_edgecolor('red')
bars[names.index(best)].set_linewidth(3)

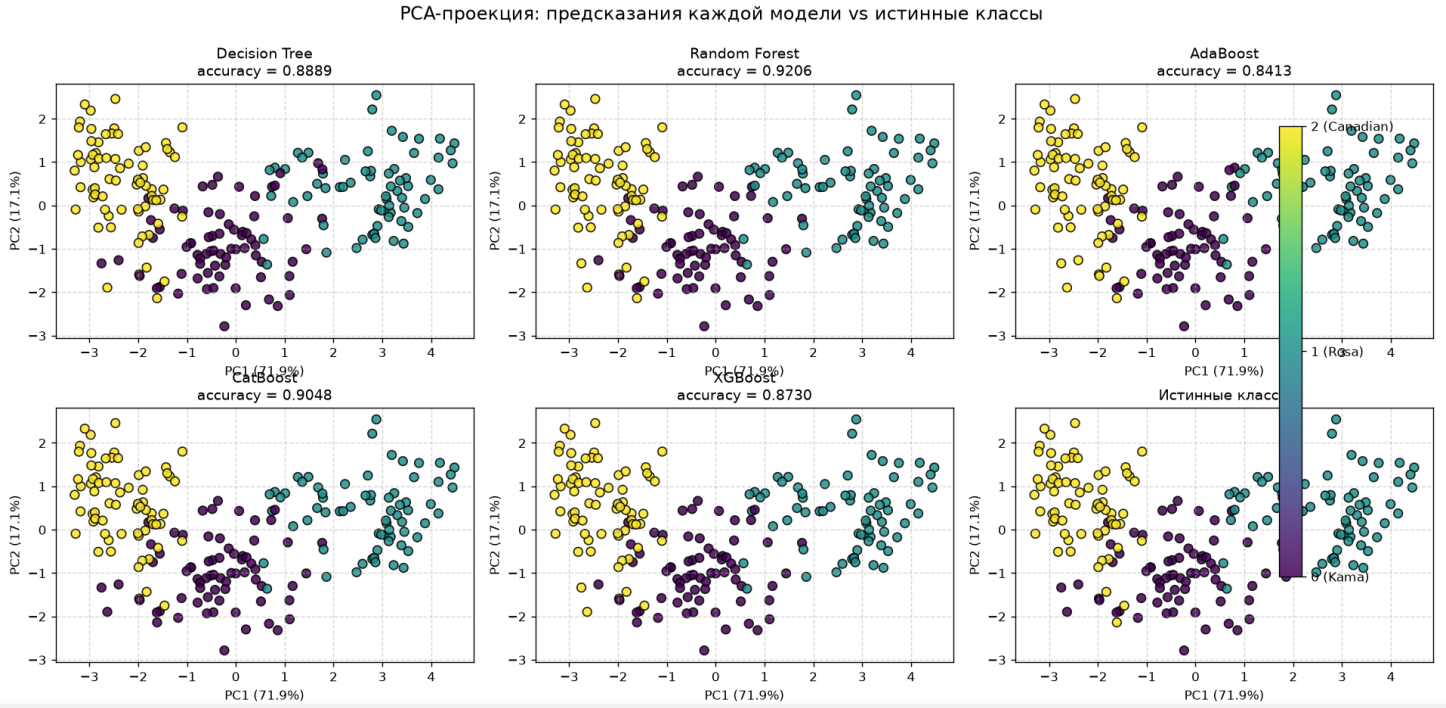
plt.ylim(0.75, 1.0)
plt.ylabel('Accuracy на тестовой выборке')
```

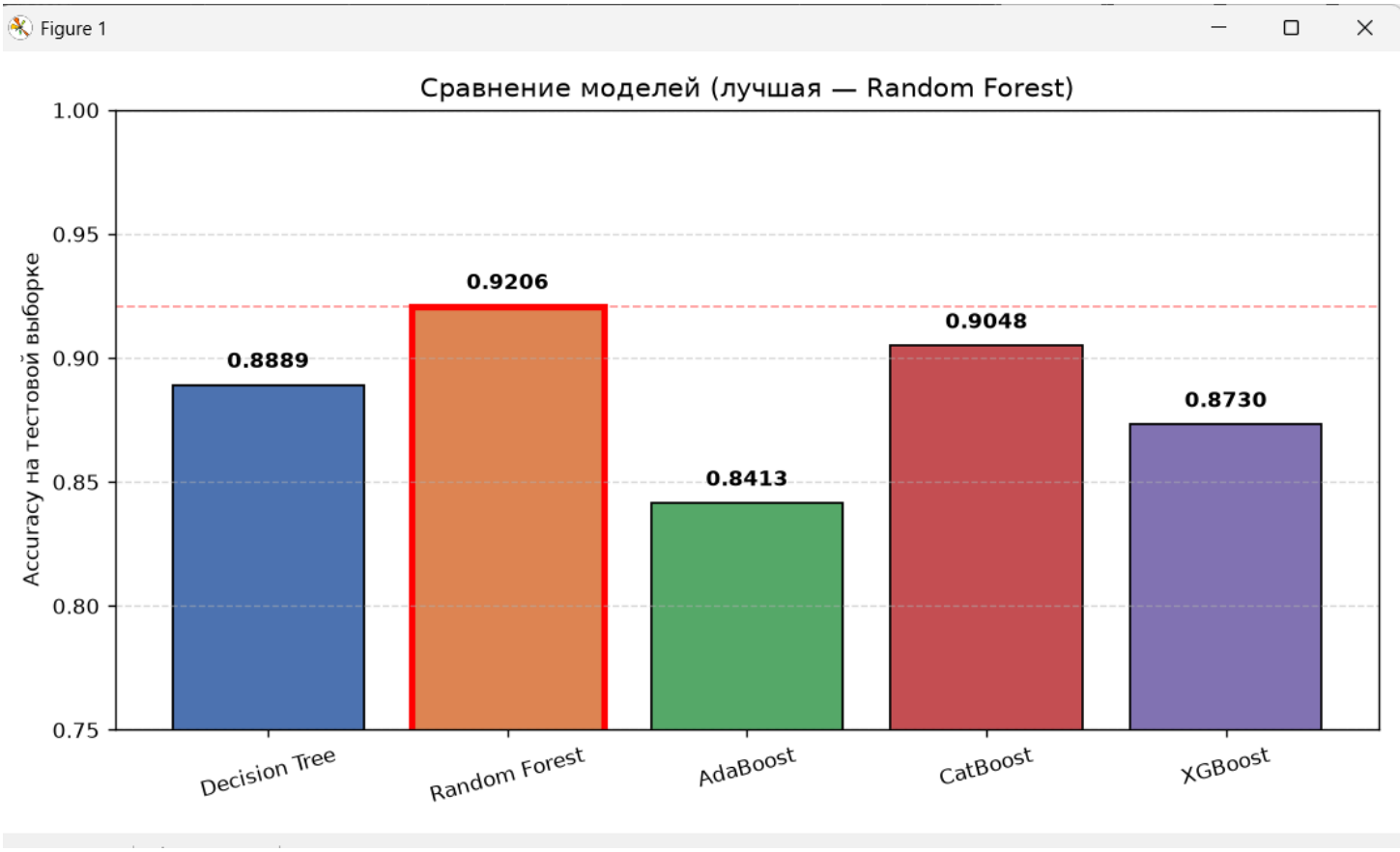
```
plt.title(f'Сравнение моделей (лучшая — {best})')
plt.axhline(y=max(accs), color='red', linestyle='--', alpha=0.4, linewidth=1)
plt.grid(True, axis='y', linestyle='--', alpha=0.5)
plt.xticks(rotation=15)
plt.tight_layout()
plt.savefig('accuracy_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
```

Результат:

C:\Users\arosl\PycharmProjects\IAD5\.venv\Scripts\python.exe
C:\Users\arosl\PycharmProjects\IAD5\main.py
Decision Tree : accuracy = 0.8889
Random Forest : accuracy = 0.9206
AdaBoost   : accuracy = 0.8413
CatBoost   : accuracy = 0.9048
XGBoost   : accuracy = 0.8730

Объяснённая дисперсия: PC1 = 0.719, PC2 = 0.171, сумма = 0.890





Вывод: В ходе лабораторной работы я обучил пять разных моделей на данных о семенах пшеницы и на практике убедился, что ансамбли работают лучше одиночных алгоритмов. Лучший результат показал случайный лес с точностью 92%, а хуже всех себя проявил AdaBoost (84%). Построенные графики PCA наглядно показали, почему так вышло: сорт «Kama» все модели легко отделяют от остальных, а вот сорта «Rosa» и «Canadian» сильно перемешаны в пространстве признаков (желтые и зеленые точки накладываются друг на друга). Случайный лес лучше всех смог «прорубить» границу между этими похожими классами, поэтому для моей задачи (где мало данных и признаки сильно коррелируют) подходит именно он.